PROJECT REPORT - CUSTOM CAT

Name: Nayan Kumar Choudhary

Admission Number: 23JE0636

Group Number: 25

---

1. Introduction

This project involves developing simplified UNIX-like utilities in C, along with a custom-built shell to execute them. The shell acts as an interface between the user and the operating system, allowing users to run commands and interact with the system.

In this project, I was responsible for implementing the custom cat utility (custom_cat), which replicates the core behavior of the standard Unix cat command. It reads and displays file contents, supports concatenation of multiple files, provides line numbering, and handles standard input.

The project provides practical exposure to system-level programming concepts such as file I/O, character-level input processing, command-line argument parsing, and error handling in C.

2. Objective

- To implement a custom version of the Unix cat utility from scratch in C
- To understand file I/O operations using fopen(), fgetc(), and fclose()
- To handle command-line arguments and flags (-n for line numbering)
- To process multiple files sequentially and concatenate their output
- To handle standard input when no file is provided
- To practice modular C programming with separate, focused functions

3. Features of custom_cat

- Display contents of one or more files to standard output
- Line numbering with the -n flag (format: ' 1 | line content')
- Concatenate multiple files and print them sequentially
- Read from standard input (stdin) when no file argument is given
- Graceful error handling for missing or unreadable files
- Continues processing remaining files even if one fails
- Returns proper exit status (0 for success, 1 if any file failed)

4. Concepts Used

- File I/O operations (fopen(), fgetc(), fclose())
- Standard input handling (stdin as a FILE pointer)
- Character-by-character reading for precise line detection
- Command-line argument parsing (argc, argv)

- Error reporting using fprintf(stderr) and perror()
- Modular function design in C
- Exit status management for shell integration

5. Code Explanation

The implementation of custom_cat is provided in custom_cat.c. The code is organized into three focused functions:

show_contents()

This is the core display function. It reads the file pointer character by character using fgetc() and prints each character using putchar(). An integer variable at_line_start tracks whether the current position is at the beginning of a new line. When the -n flag is active (flag_numbering = 1), a line number prefix is printed at the start of each line in the format ' N | '. The counter increments after each newline character is detected.

open_and_display()

This function handles safe file opening. It takes a filename and the numbering flag as parameters. It attempts to open the file using fopen() in read mode. If the file cannot be opened, it prints a descriptive error using fprintf(stderr) and perror() for OS-level detail, then returns -1 to signal failure to the caller. On success, it calls show_contents() and closes the file with fclose().

parse_flags()

This function scans the command-line arguments for supported flags. It iterates through argv while arguments begin with '-'. If -n is detected, it sets the flag_numbering variable to 1. Unknown flags trigger a warning message. The function returns the index of the first non-flag argument, which is where the file names begin.

main()

The entry point calls parse_flags() to detect options and find where file arguments start. If no file arguments remain after flag parsing, it falls back to reading from stdin by calling show_contents(stdin, flag_numbering). Otherwise, it iterates over each filename and calls open_and_display(). A global exit_status variable tracks whether any file failed to open, and is returned at the end.

Function	Description
show_contents()	Reads char by char, prints content, handles line numbering using at_line_start flag
open_and_display()	Safely opens a file, calls show_contents(), handles errors with perror()
parse_flags()	Scans argv for flags, returns index where file arguments begin
main()	Entry point, ties all functions together, handles stdin fallback

6. Execution Steps

Compile:

```
gcc -Wall -g -o custom_cat custom_cat.c
```

Run standalone:

```
./custom_cat file.txt
```

Run inside custom_shell:

```
./custom_shell
custom_cat file.txt
```

7. Usage Examples

Display a file:

```
./custom_cat test.txt
```

Display with line numbers:

```
./custom_cat -n test.txt
```

Concatenate two files:

```
./custom_cat file1.txt file2.txt
```

Read from stdin:

```
echo "hello" | ./custom_cat
```

Error handling for missing file:

```
./custom_cat fakefile.txt
```

8. Screenshots

The following screenshots show the output of custom_cat running inside the custom_shell developed by Adyant (22JE0067):

- Test 1 — Basic file display: custom_cat test.txt

```
nayan@NAYANKUMARCHY:~/G25_OS-Project/G25_Project2_1$ gcc -Wall -g -o custom_shell custom_shell.c
nayan@NAYANKUMARCHY:~/G25_OS-Project/G25_Project2_1$ ./custom_shell
Welcome to Custom Shell! Type 'help' for commands.
custom_shell:~/G25_OS-Project/G25_Project2_1$ custom_cat test.txt
Nayan is testing
This is line two
OS project G25
Final line here
```

- Test 2 — Line numbering: custom_cat -n test.txt

```
nayan@NAYANKUMARCHY:~/G25_OS-Project/G25_Project2_1$ ./custom_cat -n test.txt
    1 | Nayan is testing
    2 | This is line two
    3 | OS project G25
    4 | Final line here
```

- Test 3 — Multiple file concatenation: custom_cat test.txt test2.txt

```
nayan@NAYANKUMARCHY:~/G25_OS-Project/G25_Project2_1$ echo -e "Second file\nAnother line" > test2.txt
./custom_cat test.txt test2.txt
Nayan is testing
This is line two
OS project G25
Final line here
Second file
Another line
```

- Test 4 — Stdin input: echo "Hello from stdin" | custom_cat

```
nayan@NAYANKUMARCHY:~/G25_OS-Project/G25_Project2_1$ echo "Hello from stdin" | ./custom_cat
Hello from stdin
```

- Test 5 — Error handling: custom_cat fakefile.txt

```
nayan@NAYANKUMARCHY:~/G25_OS-Project/G25_Project2_1$ ./custom_cat fakefile.txt
custom_cat: cannot open 'fakefile.txt': No such file or directory
```

9. Conclusion

The custom_cat implementation provided hands-on experience with low-level file handling in C. By reading input character by character rather than line by line, the implementation achieves precise control over line boundary detection, which is essential for accurate line numbering.

This project strengthened my understanding of file I/O system calls, command-line argument processing, modular function design, and error handling in C. The utility integrates seamlessly with the team's custom_shell since the shell uses execvp() to resolve and run external binaries from the same directory.